

神经网络

Neural Network

孟庆鑫

人工智能学院



唐山学院
TANGSHAN UNIVERSITY

课件获取地址



GitHub



Gitee

商务智能在解决什么问题

企业每天都在提出预测问题：

- 用户会不会购买？
- 明天销量是多少？
- 哪些客户会流失？
- 哪笔交易可能欺诈？
- 给这个用户推荐什么商品？

这些问题看似不同，本质却相同：

根据已有信息，预测未来结果

商业问题的数学本质

用户信息、价格、时间、行为记录记为输入：

$$x \in \mathbb{R}^m$$

购买概率、销量、风险评分记为输出：

$$y \in \mathbb{R}^n$$

于是问题变成：

$$y \approx f(x)$$

商务智能的核心任务之一，就是学习这个函数 f

逼近函数的两种策略

- 一：整体展开

用一组简单函数叠加，整体描述目标函数

- 二：局部拼接

把空间切成许多小块，每块分别处理，再拼接起来

策略一：整体展开

给定一组基函数 $\phi_1, \phi_2, \dots, \phi_d$ ，令

$$f(x) \approx \sum_{k=0}^d \theta_k \phi_k(x)$$

学习的对象是系数：

$$\theta_0, \theta_1, \theta_2, \dots, \theta_d$$

这正是线性回归的本质形式

典型例子：多项式逼近

取基函数：

$$1, x, x^2, \dots, x^d$$

则得到

$$f(x) \approx c_0 + c_1x + c_2x^2 + \dots + c_dx^d$$

优点：

- 简单
- 光滑
- 理论成熟

典型例子：Fourier 逼近

取基函数：

$$1, \sin x, \cos x, \sin 2x, \cos 2x, \dots$$

则得到

$$f(x) \approx \sum_{k=0}^d a_k \sin(kx) + b_k \cos(kx)$$

特别适合周期结构与振荡现象

整体展开的困境

这类方法虽然优美，却有天然限制：

- 基函数必须预先选定
- 局部突变往往需要很多项配合
- 高维组合爆炸

于是，人们开始关注另一条道路

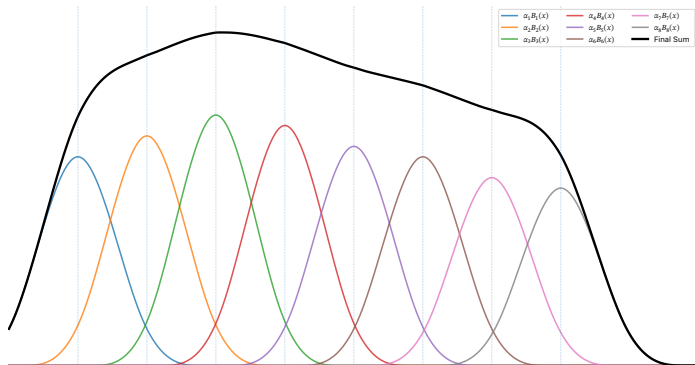
策略二：局部拼接

如果整体太复杂，那就不再试图每一次都描述全局

把空间分成很多个小片段，在每一小片段中使用简单规律

典型例子：样条函数（Spline）

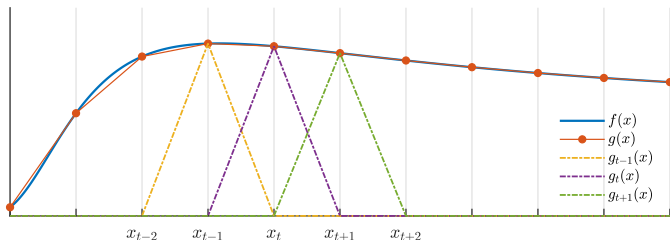
在不同区间上使用低次多项式，并在连接处保持连续光滑



复杂函数被拆成许多简单片段

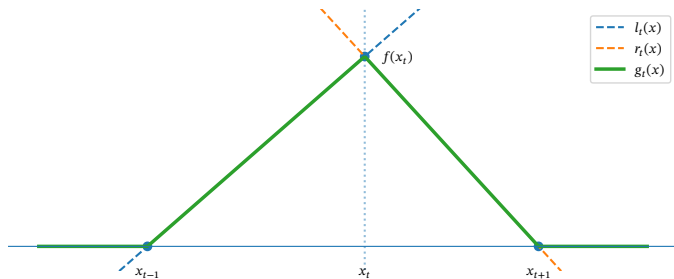
继续简化：分段线性逼近

继续追求简单，我们甚至可以只保留最基本的直线片段



$$f(x) \approx g(x) = \sum_t g_t(x)$$

局部积木：金字塔函数 g_t



$$g_t(x) = \max\{0, \min\{l_t(x), r_t(x)\}\}$$

$$l_t(x) = f(x_t) \frac{x - x_{t-1}}{x_t - x_{t-1}} \quad r_t(x) = f(x_t) \frac{x - x_{t+1}}{x_t - x_{t+1}}$$

转写 g_t

我们希望用单一函数来组合出 g_t 。引入

$$\sigma(x) = \max\{0, x\}$$

使用恒等式

$$\min\{a, b\} = a - \sigma(a - b) \quad a = \sigma(a) - \sigma(-a)$$

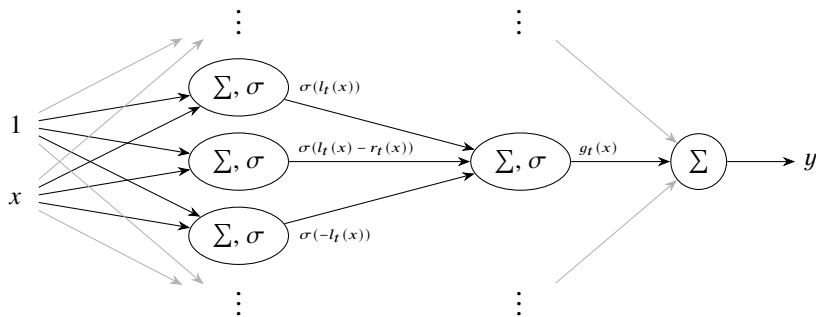
得到

$$\begin{aligned} g_t(x) &= \max\{0, \min\{l_t(x), r_t(x)\}\} \\ &= \sigma(\sigma(l_t(x)) - \sigma(-l_t(x)) - \sigma(l_t(x) - r_t(x))) \end{aligned}$$

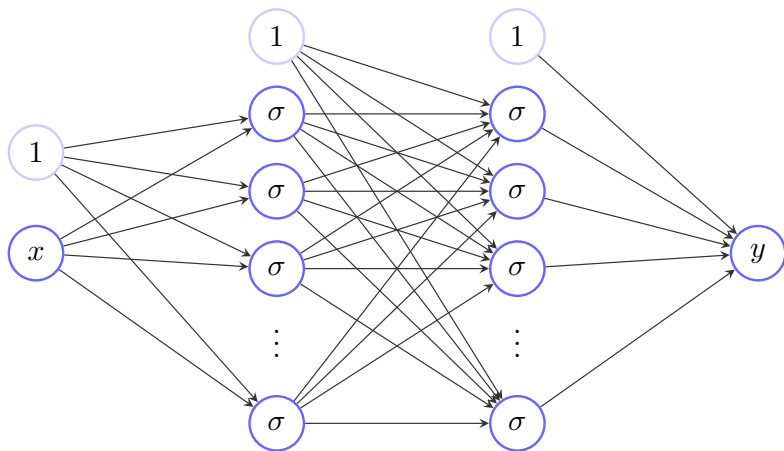
σ 被称为整流线性单元 (Rectified Linear Unit, ReLU)

局部结构

$$g_t(x) = \sigma(\sigma(l_t(x)) - \sigma(-l_t(x)) - \sigma(l_t(x) - r_t(x)))$$



结构浮现



神经网络的结构雏形已经出现

从一维走向高维

我们刚刚完成了：

单输入单输出函数的网络构造

但真实问题通常是

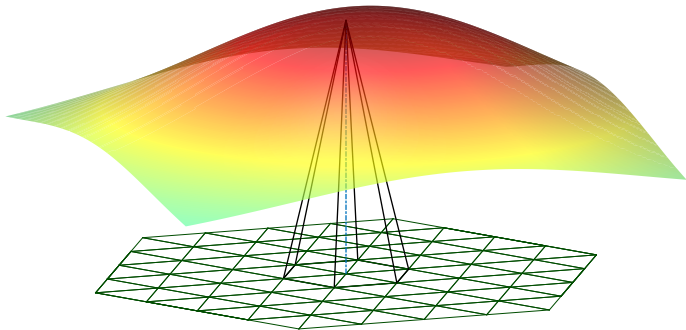
$$f : \mathbb{R}^m \rightarrow \mathbb{R}^n$$

因此，我们采用最自然的路径：

先理解双输入单输出，再推广至多输入多输出系统

二维情形：继续局部拼接

对双输入函数 $f(x)$, $x = (x_1, x_2) \in \mathbb{R}^2$, 同样可以做分片线性逼近

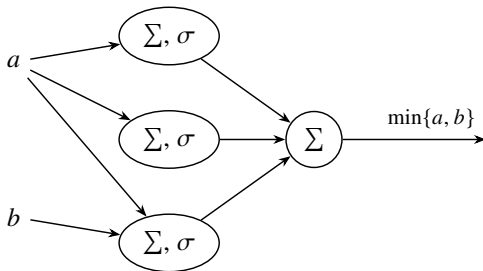


$$f(x) \approx g(x) = \sum_t g_t(x), \quad x = (x_1, x_2) \in \mathbb{R}^2$$

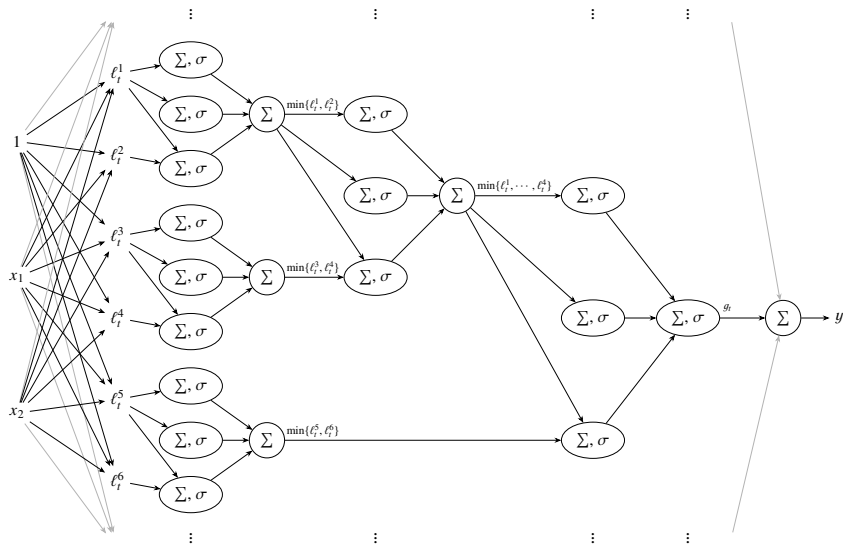
转写 g_t

$$g_t = \max \left\{ 0, \min_{i=1, \dots, 6} \ell_t^i \right\}$$
$$= \sigma \left(\min \left\{ \min \left\{ \min \{ \ell_t^1, \ell_t^2 \}, \min \{ \ell_t^3, \ell_t^4 \} \right\}, \min \{ \ell_t^5, \ell_t^6 \} \right\} \right).$$

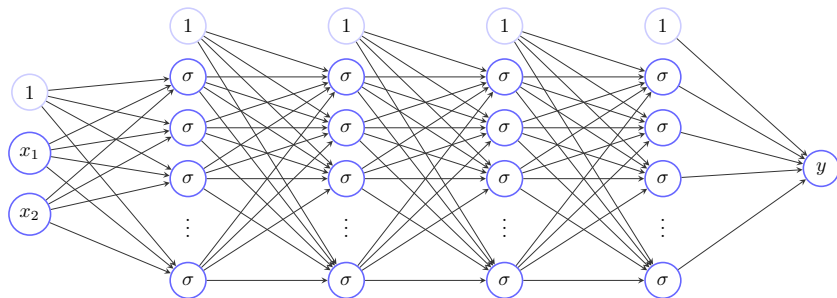
利用恒等式 $\min\{a, b\} = \sigma(a) - \sigma(-a) - \sigma(a - b)$ 对应的结构



局部结构



结构再现



网络结构再次出现

归纳多输入情形

函数 f 可近似用一组金字塔函数的组合来表示:

$$f(x) \approx g(x) = \sum_t g_t(x)$$

- x 是一维:

$$g_t(x) = \max\{0, \min\{l_t(x), r_t(x)\}\}$$

- x 是二维:

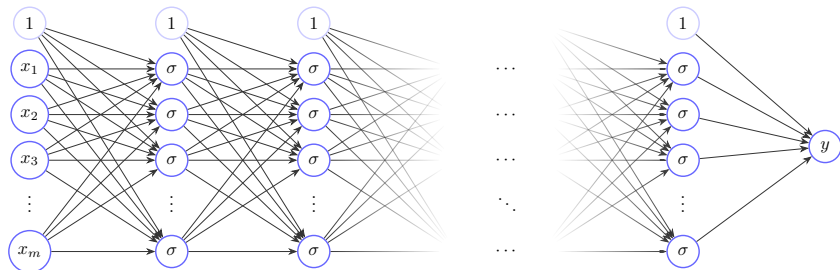
$$g_t = \max\{0, \min_{i=1, \dots, 6} \ell_t^i\}$$

- 可归纳 x 是 m 维:

$$g_t = \max\{0, \min_i \ell_t^i\}$$

m 维输入的网络结构

一个更宽、更深的网络



推广至多输入多输出系统

对于一般系统：

$$f : \mathbb{R}^m \rightarrow \mathbb{R}^n$$

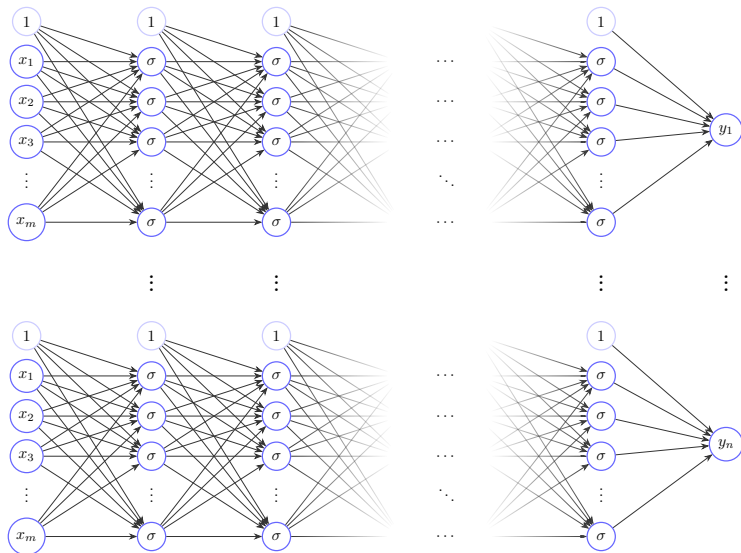
可将其按输出坐标拆解为 n 个多输入单输出子问题：

$$f(x) = \begin{bmatrix} f_1(x) \\ f_2(x) \\ \vdots \\ f_n(x) \end{bmatrix}, \quad f_j : \mathbb{R}^m \rightarrow \mathbb{R}$$

每个分量函数 f_j 都可以分别用网络逼近

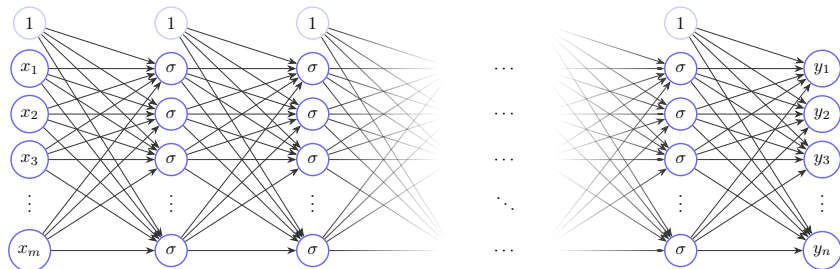
因此，只需共享输入层，并并行组合这 n 个子网络
即可得到整体的多输出网络结构

扩展至 n 维输出



现代神经网络的诞生

我们用下面的网络来逼近多输入多输出系统 $f: \mathbb{R}^m \rightarrow \mathbb{R}^n$



这就是现代全连接 ReLU 神经网络

神经网络的另一种理解：自动特征学习

我们发现

- 神经网络的输出是最后一层隐藏层的线性变换
- 线性回归是特征函数向量的线性变换

这意味着

神经网络通过前面的多层结构在自动学习特征函数

我们是否高估了深度？

- 我们为单输入（一元）系统构建了 2 隐层网络
- 我们为双输入（二元）系统构建了 4 隐层网络
- 随着输入维度的升高，网络的层数也需要相应加深

事实真的如此么？

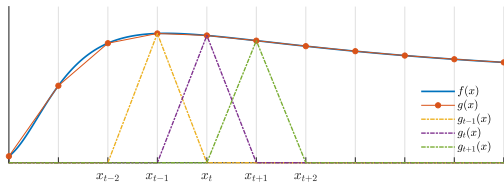
一个令人意外的事实

即使只有一层隐藏层

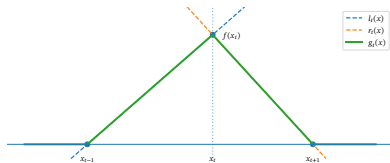
神经网络依然可以逼近极其广泛的函数

回顾单输入单输出情形

- $f(x) \approx g(x) = \sum_t g_t(x)$

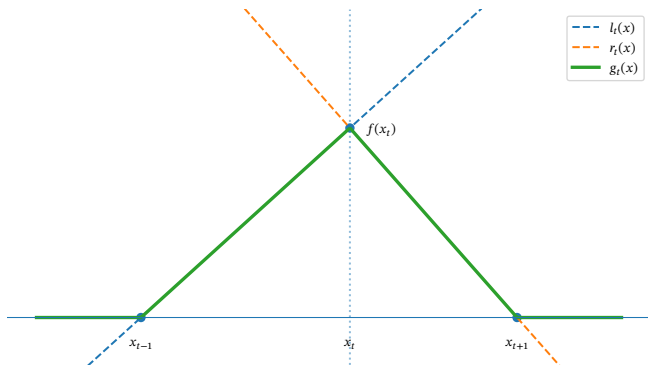


- $g_t(x) = \sigma(\sigma(l_t(x)) - \sigma(-l_t(x)) - \sigma(l_t(x) - r_t(x)))$



- 2 隐层结构源于 g_t 中存在 σ 的复合

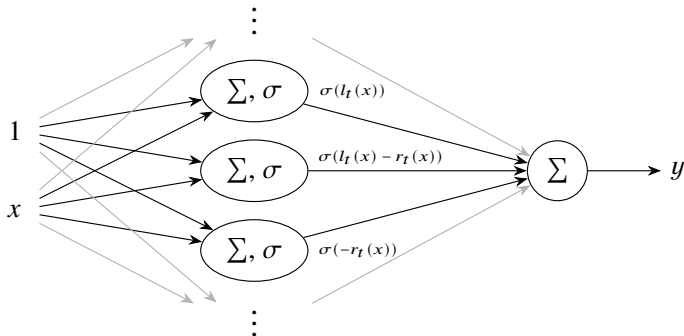
重写 g_t



$$g_t(x) = \sigma(l_t(x)) - \sigma(l_t(x) - r_t(x)) + \sigma(-r_t(x))$$

单隐层结构出现

$$g_t(x) = \sigma(l_t(x)) - \sigma(l_t(x) - r_t(x)) + \sigma(-r_t(x))$$



结论：一元函数可以用单隐层神经网络逼近

再看多输入单输出情形

在适当条件下，许多多元函数 $f: \mathbb{R}^m \rightarrow \mathbb{R}$ 可由 Fourier 形式逼近

$$f(x) \approx \sum_k a_k \sin(w_k^\top x + b_k) + c_k \cos(w_k^\top x + b_k)$$

- $w_k^\top x + b_k$ 是对输入 $x \in \mathbb{R}^m$ 的仿射变换
- $\sin$ 和 $\cos$ 是一元函数，可以用单隐层神经网络逼近
- a_k 和 c_k 是组合系数

结论：多元函数也可以用单隐层神经网络逼近

推广至多输出系统

对于一般系统 $f : \mathbb{R}^m \rightarrow \mathbb{R}^n$ ，可按坐标拆分：

$$f(x) = \begin{bmatrix} f_1(x) \\ f_2(x) \\ \vdots \\ f_n(x) \end{bmatrix}$$

每个分量 $f_j(x)$ 分别逼近，再并联组合即可

结论：多输入多输出系统也可以用单隐层神经网络逼近

单隐层 ReLU 网络的逼近能力

单隐层 ReLU 网络可逼近任意连续映射

$$f : \mathbb{R}^m \rightarrow \mathbb{R}^n$$

这意味着：

深度，并不是逼近能力存在的前提

为何要深度学习?

既然单层网络已经足够强大

为什么现代人工智能

仍然选择越来越深的网络结构?

因为

深度更高效

能力与效率

单层网络解决的是：

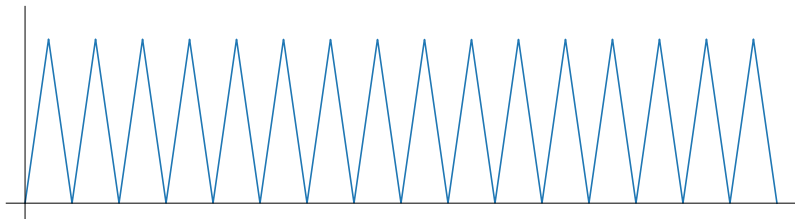
能否逼近目标函数，这是表达能力范畴

深层网络解决的是：

能否用更少资源完成逼近，这是表达效率范畴

一个简单例子：重复结构

考虑这样一个函数



这个函数有 2^L 个相同的峰值结构

若使用单层网络

若直接用单隐层网络表示 2^L 个峰值结构

需要隐藏层平行构造 2^L 个金字塔函数

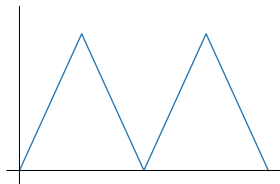
因为每一个局部峰都需要单独摆放

这意味着

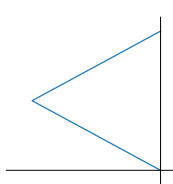
$$\text{规模} \propto 2^L$$

若使用深层网络

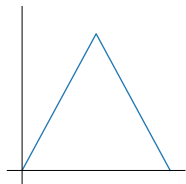
每一层都构建一个金字塔函数，我们归纳 $L + 1$ 层堆叠的结果



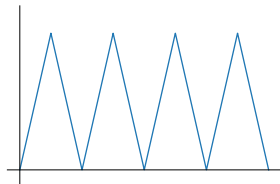
第 2 层的结果



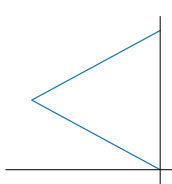
第 2 层



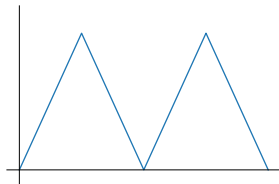
第 1 层



第 3 层的结果



第 3 层



第 2 层的结果

若使用深层网络

每一层的结果表现为“复制一次结构”

1 峰 $\longrightarrow$ 2 峰 $\longrightarrow$ 4 峰 $\longrightarrow$ 8 峰 $\longrightarrow \dots$

经过 $L + 1$ 层后即可产生 2^L 个峰值结构

这意味着

规模 $\propto L$

深度的价值

深度创造了强大的结构复用能力

上一层生成的结果

可被下一层再次调用、再次组合

理解宽度与深度

增加宽度，意味着：

有能力逼近更复杂的结构

增加深度，意味着：

高效复用已有结构

二者都重要，但侧重不同

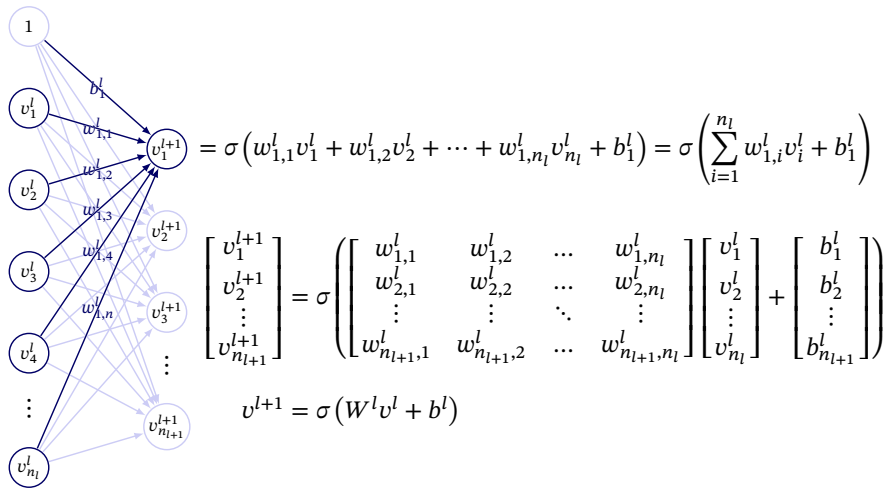
神经网络的参数化

前面我们讨论的是网络结构：节点如何连接、层如何排列
但同样的结构，若连接强弱不同，输出结果也会完全不同

结构决定表达形式

参数决定具体行为

两层之间的映射关系



多层网络的递推表达

对 L 个隐藏层的神经网络，设输入为

$$v^0 = x$$

第 l 个隐藏层的输出由前一层产生：

$$v^l = \sigma(W^{l-1}v^{l-1} + b^{l-1}), \quad l = 1, 2, \dots, L$$

最终输出：

$$\hat{y} = W^L v^L + b^L$$

整张网络，本质是仿射变换与非线性映射的层层复合

从参数化走向训练

神经网络可被写成参数化映射

$$\hat{y} = h(x; W^0, b^0, \dots, W^L, b^L)$$

下面的问题自然变成

如何确定这些矩阵 W^l 与偏置 b^l ，使预测尽可能准确？

神经网络的岭回归目标

延续线性回归中的思想，考虑：

$$\mathcal{J} = \frac{1}{2N} \sum_{i=1}^N \|\hat{y}_i - y_i\|^2 + \frac{\lambda}{2} \sum_{l=0}^L (\|W^l\|_F^2 + \|b^l\|^2)$$
$$\text{s.t.} \quad \begin{cases} \hat{y}_i = W^L v_i^L + b^L \\ v_i^l = \sigma(W^{l-1} v_i^{l-1} + b^{l-1}), & l = 1, 2, \dots, L \\ v_i^0 = x_i \end{cases}$$

其中：

- 第一项：拟合训练样本
- 第二项：控制参数规模

求解神经网络岭回归的方法

不同于线性回归可直接求解析解，神经网络通常采用迭代优化方法
这是因为神经网络

- 含多层非线性激活函数 σ
- 参数数量巨大（矩阵很多）
- 目标函数通常非凸
- 难以写出显式最优解

我们选择随机梯度下降（Stochastic Gradient Descent, SGD）
以降低计算代价

用 SGD 求解神经网络的岭回归

假设第 t 轮迭代随机选择了样本 (x_i, y_i) , 则 SGD 更新公式为

$$W_{t+1}^l \leftarrow W_t^l - \eta \frac{\partial \mathcal{J}_i}{\partial W_t^l}$$
$$b_{t+1}^l \leftarrow b_t^l - \eta \frac{\partial \mathcal{J}_i}{\partial b_t^l}$$

其中

$$\mathcal{J}_i = \frac{1}{2} \|\hat{y}_i - y_i\|^2 + \frac{\lambda}{2} \sum_{l=0}^L (\|W^l\|_F^2 + \|b^l\|^2)$$

实现 SGD 的关键在于求解 $\mathcal{J}_i$ 对各参数的偏导数

我们可以通过对 $\mathcal{J}_i$ 的全微分来一次求出

从全微分得到耦合方程组

对单样本目标函数 $\mathcal{J}_i$ 和神经网络递推式分别取全微分，得到

$$d\mathcal{J}_i = (\hat{y}_i - y_i)^T d\hat{y}_i + \lambda \sum_{l=0}^L (\langle W^l, dW^l \rangle + (b^l)^T db^l)$$

$$d\hat{y}_i = dW^L v_i^L + W^L dv_i^L + db^L$$

$$dv_i^l = \sigma'(z_i^l) \odot dz_i^l, \quad l = 1, \dots, L$$

$$dz_i^l = dW^{l-1} v_i^{l-1} + W^{l-1} dv_i^{l-1} + db^{l-1}, \quad l = 1, \dots, L$$

这构成了一组关于 $d\hat{y}_i$, dv_i^l , dz_i^l , dW^l , db^l 的一阶线性化耦合系统

求解线性耦合系统

求解一阶线性化耦合系统得到

$$\frac{\partial \mathcal{J}_i}{\partial W^l} = \delta_i^{l+1} (v_i^l)^T + \lambda W^l$$
$$\frac{\partial \mathcal{J}_i}{\partial b^l} = \delta_i^{l+1} + \lambda b^l$$

其中

$$\delta_i^{L+1} \leftarrow \hat{y}_i - y_i$$
$$\text{for } l = L, L-1, \dots, 1 :$$
$$\delta_i^l \leftarrow (W^l)^\top \delta_i^{l+1} \odot \sigma'(z_i^l)$$

这就是神经网络训练中的反向传播算法 (Backpropagation)

深层网络的数值危机

深度神经网络本质上是大量映射的连续复合：

$$f \approx f_L \circ f_{L-1} \circ \cdots \circ f_1$$

其中每一层：

$$f_l(v) = \sigma(W^l v + b^l)$$

复合结构虽然带来了表达效率，但同时也意味着：

数值必须连续 “穿过层层映射”

大量映射复合之后，数值传播仍然稳定么？

深层网络的局部传播

对网络递推式

$$v^l = \sigma(W^{l-1}v^{l-1} + b^{l-1})$$

取微分：

$$dv^l = D^l W^{l-1} dv^{l-1}$$

其中

$$D^l = \text{diag}(\sigma'(z^l))$$

继续递推：

$$d\hat{y} = W^L \left(\prod_{l=1}^L D^l W^{l-1} \right) dx$$

这意味着：输入的微小扰动会经历大量矩阵连乘传递至输出

指数尺度效应

- 若每层局部传播都略微缩小，即 $\|D^l W^{l-1}\| < 1$ ，则：

$$\left\| \prod_{l=1}^L D^l W^{l-1} \right\| \rightarrow 0$$

这意味着输入的微小扰动将被网络淹没，无法传递给输出

- 若每层局部传播都略微放大，即 $\|D^l W^{l-1}\| > 1$ ，则：

$$\left\| \prod_{l=1}^L D^l W^{l-1} \right\| \rightarrow \infty$$

这意味着输入的微小扰动将被网络放大，导致输出的巨大改变

数值稳定性问题贯穿网络训练全过程

在神经网络的初始化阶段及训练全过程中，若局部传播：

$$dv^l = D^l W^{l-1} dv^{l-1}$$

持续发生缩小或放大，则微小扰动会在深层传播中被指数衰减，或被指数放大

因此，我们要在网络训练全过程中持续控制局部尺度的稳定性，即：

$$\mathbb{E}\|dv^l\|^2 \approx \mathbb{E}\|dv^{l-1}\|^2$$

Gaussian 初始化下的随机微扰传播

前向传播满足：

$$dv_i^l = \sum_{j=1}^{n_{l-1}} (D^l W^{l-1})_{ij} dv_j^{l-1}$$

其中 n_{l-1} 表示第 $l-1$ 层网络包含的节点（神经元）数量

若初始化满足 $W_{ij}^{l-1} \sim \mathcal{N}(0, \sigma_{l-1}^2)$ ，则随机微扰尺度近似满足：

$$\begin{aligned}\mathbb{E}\|dv^l\|^2 &\approx n_{l-1} \mathbb{E}[(\sigma'(z^l))^2] \sigma_{l-1}^2 \mathbb{E}\|dv^{l-1}\|^2 \\ &\approx \frac{1}{2} n_{l-1} \sigma_{l-1}^2 \mathbb{E}\|dv^{l-1}\|^2\end{aligned}$$

若初始化后的 z^l 近似关于 0 对称，则约有一半节点被 ReLU 激活

He 初始化

我们希望：

$$\mathbb{E}\|\mathbf{d}v^l\|^2 \approx \mathbb{E}\|\mathbf{d}v^{l-1}\|^2$$

这意味着 $\frac{1}{2} n_{l-1} \sigma_{l-1}^2 \approx 1$ ，即

$$\sigma_{l-1}^2 \approx \frac{2}{n_{l-1}}$$

因此参数初始化应满足

$$w_{ij}^{l-1} \sim \mathcal{N}\left(0, \frac{2}{n_{l-1}}\right)$$

这就是 He 初始化

He 初始化并不能长期维持稳定

He 初始化只能保证“起点”的尺度近似稳定

训练开始后：

- 参数 $W^l \leftarrow W^l - \eta \frac{\partial \mathcal{J}_i}{\partial W^l}$ 不断更新
- 各层分布持续变化
- 深层尺度重新发生漂移

训练过程中仍可能重新出现数值不稳定

为此，我们希望有另外一套机制确保训练过程中近似保持

$$\mathbb{E}\|dv^l\|^2 \approx \mathbb{E}\|dv^{l-1}\|^2$$

加入 Batch Normalization

在网络内部加入标准化 (BatchNorm):

$$\text{BN}_l(z^l) = \gamma^l \frac{z^l - \mu_{z^l}}{\sqrt{\sigma_{z^l}^2 + \varepsilon}} + \beta^l$$

其中 μ_{z^l} 、 $\sigma_{z^l}^2$ 分别表示 z^l 的均值与方差。于是网络变为:

$$v^l = \sigma(\text{BN}_l(W^{l-1}v^{l-1} + b^{l-1})) = \sigma\left(\gamma^l \frac{W^{l-1}}{\sqrt{\sigma_{z^l}^2 + \varepsilon}} v^{l-1} + \gamma^l \frac{b^{l-1} - \mu_{z^l}}{\sqrt{\sigma_{z^l}^2 + \varepsilon}} + \beta^l\right)$$

它可写成新的参数形式 $v^l = \sigma(\widetilde{W}^{l-1}v^{l-1} + \widetilde{b}^{l-1})$

BatchNorm 后的局部尺度关系

新的局部尺度 $\mathbf{d}v^l = \tilde{D}^l \widetilde{W}^{l-1} \mathbf{d}v^{l-1} = \tilde{D}^l \frac{\gamma^l W^{l-1}}{\sqrt{\sigma_{z^l}^2 + \varepsilon}} \mathbf{d}v^{l-1}$

于是局部尺度满足：

$$\mathbb{E}\|\mathbf{d}v^l\|^2 \approx n_{l-1} \mathbb{E}[(\sigma'(z_{\text{BN}}^l))^2] \frac{(\gamma^l)^2 \text{Var}(W^{l-1})}{\sigma_{z^l}^2 + \varepsilon} \mathbb{E}\|\mathbf{d}v^{l-1}\|^2$$

在独立性近似下： $\sigma_{z^l}^2 \approx n_{l-1} \text{Var}(W^{l-1}) \text{Var}(v^{l-1})$

将其代入局部尺度关系，得到

$$\mathbb{E}\|\mathbf{d}v^l\|^2 \approx \mathbb{E}[(\sigma'(z_{\text{BN}}^l))^2] \frac{(\gamma^l)^2}{\text{Var}(v^{l-1})} \mathbb{E}\|\mathbf{d}v^{l-1}\|^2$$

ReLU + BatchNorm 的粗略尺度分析

对于 ReLU:

$$\sigma'(x) = \begin{cases} 1, & x > 0 \\ 0, & x < 0 \end{cases}$$

因此:

$$\mathbb{E}[(\sigma'(z_{\text{BN}}^l))^2] = \mathbb{P}(z_{\text{BN}}^l > 0)$$

BatchNorm 会使:

$$z_{\text{BN}}^l \approx \mathcal{N}(\beta^l, (\gamma^l)^2)$$

于是:

$$\mathbb{E}[(\sigma'(z_{\text{BN}}^l))^2] \approx \Phi\left(\frac{\beta^l}{\gamma^l}\right)$$

其中 Φ 表示高斯分布函数

ReLU 输出方差的粗略尺度分析

注意到

$$z_{\text{BN}}^{l-1} \approx \mathcal{N}(\beta^{l-1}, (\gamma^{l-1})^2)$$

于是

$$v^{l-1} = \sigma(z_{\text{BN}}^{l-1})$$

成为截断高斯随机变量

当 $\beta^{l-1} \approx 0$ 时:

$$\text{Var}(v^{l-1}) \propto (\gamma^{l-1})^2$$

尺度因子的进一步化简

整理得到：

$$\begin{aligned}\mathbb{E}\|\mathrm{d}v^l\|^2 &\approx \mathbb{E}[(\sigma'(z_{\text{BN}}^l))^2] \frac{(\gamma^l)^2}{\text{Var}(v^{l-1})} \mathbb{E}\|\mathrm{d}v^{l-1}\|^2 \\ &\approx C \left(\frac{\gamma^l}{\gamma^{l-1}} \right)^2 \mathbb{E}\|\mathrm{d}v^{l-1}\|^2\end{aligned}$$

其中 C 接近一个稳定常数

继续递推：

$$\mathbb{E}\|\mathrm{d}v^L\|^2 \approx C^L \left(\frac{\gamma^L}{\gamma^0} \right)^2 \mathbb{E}\|\mathrm{d}x\|^2$$

BatchNorm 对指数累积的缓解

$$\mathbb{E}\|dv^L\|^2 \approx C^L \left(\frac{\gamma^L}{\gamma^0}\right)^2 \mathbb{E}\|dx\|^2$$

- BatchNorm 显著削弱了层间尺度的指数累积
- C^L 仍然存在，它有可能被 $\left(\frac{\gamma^L}{\gamma^0}\right)^2$ 削弱

BatchNorm 提供了一种“有望维持尺度稳定”的机制

BatchNorm 与反向传播

BatchNorm 在网络内部插入了新的映射： $z^l \mapsto \gamma^l \frac{z^l - \mu_{z^l}}{\sqrt{\sigma_{z^l}^2 + \varepsilon}} + \beta^l$

虽然网络计算过程发生了改变，但整体仍然是

仿射变换 $\longrightarrow$ ReLU $\longrightarrow$ 仿射变换 $\longrightarrow$ ReLU ...

的层层复合结构

因此，前面基于全微分建立的一阶线性化耦合系统方法仍然成立

不同之处在于：

- BatchNorm 为网络引入了新的可学习参数 γ^l 、 β^l
- 它们同样需要通过反向传播进行更新
- 其中 γ^l 会直接影响局部尺度传播

前向传播与反向传播的统一结构

反向传播满足：

$$\delta^l = (W^l)^\top \delta^{l+1} \odot \sigma'(z^l)$$

即：

$$\delta^l = (D^l W^l)^\top \delta^{l+1}$$

继续递推：

$$\delta^0 = \left(\prod_{l=1}^L D^l W^l \right)^\top \delta^L$$

前向传播与反向传播本质上都由大量矩阵连乘构成

梯度消失与梯度爆炸

- 若每层局部传播都略微缩小，即 $\|D^l W^l\| < 1$ ，则：

$$\left\| \left(\prod_{l=1}^L D^l W^l \right)^\top \delta^L \right\| \rightarrow 0$$

这意味着输入附近的参数几乎无法得到有效更新

- 若每层局部传播都略微放大，即 $\|D^l W^l\| > 1$ ，则：

$$\left\| \left(\prod_{l=1}^L D^l W^l \right)^\top \delta^L \right\| \rightarrow \infty$$

这意味着浅层网络的梯度会被指数放大

ReLU 网络中的神经元死亡

对于 ReLU 函数 $\sigma(x) = \max\{0, x\}$ ，其导数满足：

$$\sigma'(x) = \begin{cases} 1, & x > 0 \\ 0, & x < 0 \end{cases}$$

若某个神经元长期满足：

$$z_i^l < 0$$

则：

$$\sigma'(z_i^l) = 0$$

于是：

$$\delta_i^l = 0$$

这意味着该神经元将无法继续获得梯度更新

神经元死亡的连锁影响

若大量神经元进入

$$z_i^l < 0$$

区域，则：

- 大量梯度被截断
- 网络表达能力下降
- 深层训练变得困难

BatchNorm 对梯度传播的帮助

反向传播满足：

$$\delta^l = (W^l)^\top \delta^{l+1} \odot \sigma'(z^l)$$

根据前面的推导，BatchNorm 会使：

$$\mathbb{P}(z_{\text{BN}}^l > 0) = \mathbb{E}[(\sigma'(z_{\text{BN}}^l))^2] \approx \Phi\left(\frac{\beta^l}{\gamma^l}\right)$$

这意味着每层激活的节点（神经元）会保持在较稳定水平

更深层的问题

即使：

- 使用 He 初始化
- 使用 BatchNorm

深层网络中的大量矩阵连乘仍然存在：

$$\text{前向传播: } \prod_{l=1}^L D^l W^{l-1}, \quad \text{反向传播: } \prod_{l=1}^L D^l W^l$$

于是一个更根本的问题出现：

能否减少这种深层连乘带来的指数累积？

如何削弱指数累积?

问题的根源在于大量矩阵的连续连乘。以前向传播为例：

$$\prod_{l=1}^L D^l W^{l-1}$$

考虑每层都略微缩小的情形： $\|D^l W^{l-1}\| < 1$ ，这可能导致指数衰减

若将 $D^l W^{l-1}$ 替换成 $I + D^l W^{l-1}$ ，那么即使 $D^l W^{l-1} \rightarrow 0$ ，仍可保证

$$\left\| \prod_{l=1}^L D^l W^{l-1} \right\| \rightarrow 1$$

这条稳定传播路径

新的网络递推式

注意到网络递推式

$$v^l = \sigma(W^{l-1}v^{l-1} + b^{l-1})$$

及其微分式

$$dv^l = D^l W^{l-1} dv^{l-1}$$

若将 $D^l W^{l-1}$ 替换成 $I + D^l W^{l-1}$ ，这意味着新的微分式为

$$dv^l = (I + D^l W^{l-1}) dv^{l-1}$$

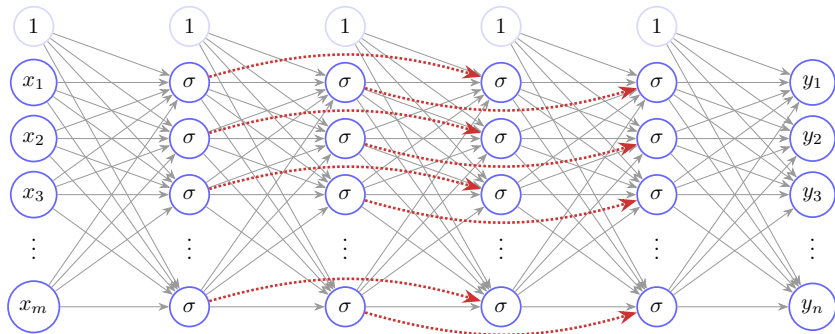
对应的网络递推式为

$$v^l = v^{l-1} + \sigma(W^{l-1}v^{l-1} + b^{l-1})$$

需要注意的是，这一递推式要求隐层节点数量都相等

新的网络结构：残差网络 (ResNet)

递推式 $v^l = v^{l-1} + \sigma(W^{l-1}v^{l-1} + b^{l-1})$ 对应于残差网络 (ResNet)



需要注意的是，ResNet 并未改变全连接 ReLU 结构

残差结构的梯度反向传播

ResNet 的反向传播满足：

$$\delta^l = (I + (D^l W^l)^\top) \delta^{l+1}$$

即使：

$$D^l W^l \rightarrow 0$$

仍有：

$$\delta^l \approx \delta^{l+1}$$

这意味着梯度可以沿恒等路径持续传播

现代深层网络的核心思想

- He 初始化：控制初始尺度
- BatchNorm：动态调节传播尺度
- ResNet：增加恒等传播路径

它们共同解决的问题是：

深层复合映射中的尺度稳定性

激活函数 ReLU 的问题

在前述反向传播推导中, $\sigma(x) = \text{ReLU}(x) = \max(0, x)$

然而 ReLU

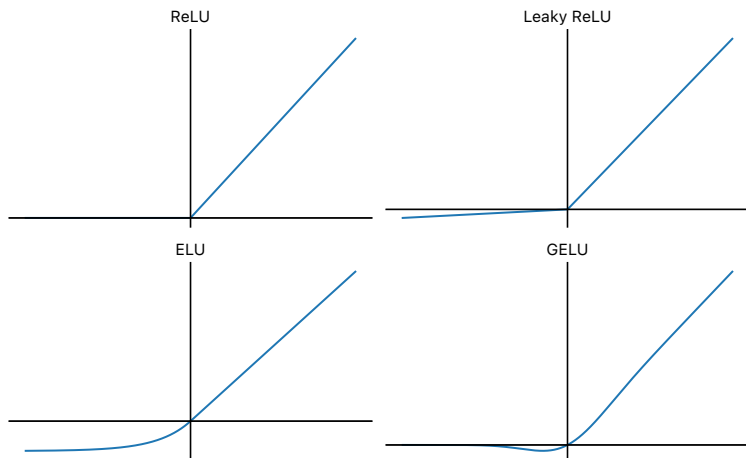
- 在 $x = 0$ 处不可导
- 在 $x < 0$ 时 $\sigma'(x) = 0$

这导致反向传播存在结构性问题:

- 神经元可能“死亡”(Dead ReLU)
- 梯度在负半轴完全阻断
- 深层网络中信息流受限

工程上会对 ReLU 进行改进, 以改善梯度传播性质

ReLU 及其改进型激活函数



SGD 的结构性局限

标准 SGD 更新规则隐含三个结构性假设：

- 所有参数共享同一学习率
- 当前梯度等价于“最优下降方向”
- 梯度尺度对所有维度同质

这些假设在深度网络中一般不成立：

- 梯度各维尺度差异显著
- 优化路径呈强噪声扰动
- 曲率方向高度各向异性

SGD 本质是“各向同性的一阶下降”

Adam：结构替换式更新规则

Adam (Adaptive Moment Estimation) 结合了：

- 动量法：平滑梯度方向
- 自适应学习率：按梯度尺度调整步长

梯度稳定时走快一些，震荡较大时走慢一些

Adam 在深度网络中表现异常稳定
已成为深度学习领域使用最广泛的优化器

我们是否过于神话 Adam 了？

Adam 真的是因为算法本身最优
才成为“默认选择”的吗？

神经网络与 Adam 优化器可能发生了隐式协同演化

一种演化视角：网络与 Adam 的协同适配

观察表明：

- 在 ResNet、Transformer 中，Adam 系列通常显著优于 SGD
- 但在某些“为特定优化器设计”的结构上，这种优势并不稳定

性能差异可能不只来自优化器本身，而来自深度学习中的闭环：

- 新结构提出 → 默认使用 Adam 训练
- 若在 Adam 下有效 → 被保留并传播
- 社区复现亦沿用 Adam
- 形成稳定的训练范式依赖

能够在 Adam 下稳定优化的结构，更容易进入主流模型库